

Методичка по написанию Unit-тестов на фреймворке JUnit 5

2 апреля 2026 г.

СОДЕРЖАНИЕ

1	Введение	3
1.1	Цель методички	3
1.2	Что такое юнит-тестирование?	3
1.3	Пирамида тестирования	3
2	Настройка проекта	4
2.1	Зависимости для Maven	4
2.2	Зависимости для Gradle	4
3	Структура теста: AAA	6
4	Основные аннотации JUnit 5	7
4.1	Пример использования аннотаций	7
5	Ассерты: проверка результатов	9
5.1	Стандартные ассерты JUnit 5	9
5.2	AssertJ: более читаемые ассерты	9
6	Параметризованные тесты	11
6.1	Подключение	11
6.2	Аннотация @ParameterizedTest	11
6.3	Источники данных для параметров	11
6.4	Пример с @MethodSource для сложных объектов	11
7	Лучшие практики написания тестов	13
7.1	Именованые тестов	13
7.2	Один тест — одна проверка	13
7.3	Используйте @BeforeEach для общей подготовки	13
7.4	Изолируйте тесты от внешних зависимостей	14
7.5	Пишите тесты до или параллельно с кодом (TDD)	14
7.6	Поддерживайте тесты в актуальном состоянии	15
8	Типичные ошибки и как их избежать	16
9	Заключение	17
10	Дополнительные ресурсы	18

1 ВВЕДЕНИЕ

1.1 Цель методички

Данный документ предназначен для разработчиков и тестировщиков, которые начинают работать с автоматизированным тестированием на языке Java. После изучения материала вы сможете:

- Понимать принципы модульного (unit) тестирования;
- Настраивать проект для работы с JUnit 5;
- Писать читаемые и поддерживаемые тесты;
- Использовать параметризованные тесты для покрытия множества ситуаций;
- Применять лучшие практики написания тестов.

1.2 Что такое юнит-тестирование?

Модульное-тестирование — это процесс определения несоответствия поведения отдельных модулей исходного кода программы заявленной спецификации.

Идея состоит в том, чтобы писать тесты для каждой нетривиальной функции или метода. Это позволяет быстро обнаружить и устранить ошибку, а также проверить, не привело ли очередное изменение кода к регрессионной ошибке - дефекту, появившемуся в ранее работающем коде после изменений.

1.3 Пирамида тестирования

- **Модульные-тесты** (основа) — тестируют отдельные классы/методы. Быстрые, изолированные, их должно быть больше всего.
- **Интеграционные тесты** — проверяют взаимодействие нескольких компонентов.
- **Е2Е-тесты** (вершина) — имитируют поведение пользователя, проверяют всю систему целиком.

JUnit 5 — это фреймворк для написания **модульных-тестов**.

2 НАСТРОЙКА ПРОЕКТА

Для добавления фреймворка JUnit5 в проект, потребуется подключить зависимости ниже.

2.1 Зависимости для Maven

Добавьте в pom.xml:

```
1  <dependencies>
2      <!-- JUnit 5 API -->
3      <dependency>
4          <groupId>org.junit.jupiter</groupId>
5          <artifactId>junit-jupiter-api</artifactId>
6          <version>5.10.0</version>
7          <scope>test</scope>
8      </dependency>
9
10     <!-- JUnit 5 Engine для запуска тестов -->
11     <dependency>
12         <groupId>org.junit.jupiter</groupId>
13         <artifactId>junit-jupiter-engine</artifactId>
14         <version>5.10.0</version>
15         <scope>test</scope>
16     </dependency>
17
18     <!-- Параметризованные тесты (опционально) -->
19     <dependency>
20         <groupId>org.junit.jupiter</groupId>
21         <artifactId>junit-jupiter-params</artifactId>
22         <version>5.10.0</version>
23         <scope>test</scope>
24     </dependency>
25
26     <!-- AssertJ для удобных ассертов (рекомендуется) -->
27     <dependency>
28         <groupId>org.assertj</groupId>
29         <artifactId>assertj-core</artifactId>
30         <version>3.24.2</version>
31         <scope>test</scope>
32     </dependency>
33 </dependencies>
```

Листинг 1: Зависимости JUnit 5 для Maven

2.2 Зависимости для Gradle

Добавьте в build.gradle:

```
1  dependencies {
2      testImplementation 'org.junit.jupiter:junit-jupiter-api:5.10.0'
3      testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.10.0'
4      testImplementation 'org.junit.jupiter:junit-jupiter-params:5.10.0'
5      testImplementation 'org.assertj:assertj-core:3.24.2'
6  }
7
```

```
8  test {  
9      useJUnitPlatform()  
10 }
```

Листинг 2: Зависимости JUnit 5 для Gradle

3 СТРУКТУРА ТЕСТА: AAA

Каждый модульный-тест следует паттерну **Arrange-Act-Assert** (Подготовка-Действие-Проверка):

```
1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  class CalculatorTest {
5
6      @Test
7      void sum_TwoPositiveNumbers_ReturnsCorrectResult() {
8          // Arrange (Подготовка)
9          Calculator calculator = new Calculator();
10         int a = 2;
11         int b = 5;
12
13         // Act (Действие)
14         int result = calculator.sum(a, b);
15
16         // Assert (Проверка)
17         assertEquals(7, result);
18     }
19 }
```

Листинг 3: Пример теста с паттерном AAA

- **Arrange**: создаём объекты, подготавливаем данные, настраиваем окружение.
- **Act**: вызываем тестируемый метод.
- **Assert**: проверяем, что результат соответствует ожиданиям.

4 ОСНОВНЫЕ АННОТАЦИИ JUNIT 5

Аннотация	Назначение
@Test	Обозначает метод как тестовый. Запускается JUnit.
@DisplayName("Описание")	Задаёт читаемое имя теста для отчётов.
@BeforeEach	Выполняется перед <i>каждым</i> тестом в классе.
@AfterEach	Выполняется после <i>каждого</i> теста в классе.
@BeforeAll	Выполняется один раз перед всеми тестами (метод должен быть static).
@AfterAll	Выполняется один раз после всех тестов (метод должен быть static).
@Disabled	Временно отключает тест (не будет запускаться).

Таблица 1: Основные аннотации JUnit 5

4.1 Пример использования аннотаций

```
1  @DisplayName("Тесты класса UserService")
2  class UserServiceTest {
3
4      private UserService userService;
5      private UserRepository mockRepository;
6
7      @BeforeAll
8      static void initAll() {
9          // Выполняется один раз перед всеми тестами
10         System.out.println("Инициализация ресурсов");
11     }
12
13     @BeforeEach
14     void setUp() {
15         // Выполняется перед каждым тестом
16         mockRepository = Mockito.mock(UserRepository.class);
17         userService = new UserService(mockRepository);
18     }
19
20     @Test
21     @DisplayName("Создание пользователя с валидными данными")
22     void createUser_WithValidData_ReturnsSuccess() {
23         // Arrange
24         String email = "test@example.com";
25
26         // Act
27         boolean result = userService.createUser(email);
28
29         // Assert
30         assertTrue(result);
31     }
32
33     @AfterEach
34     void tearDown() {
35         // Очистка после каждого теста
36         userService = null;
```

```
37     }  
38  
39     @AfterAll  
40     static void tearDownAll() {  
41         System.out.println("Очистка ресурсов");  
42     }  
43 }
```

Листинг 4: Пример использования аннотаций жизненного цикла

5 АСЕРТЫ: ПРОВЕРКА РЕЗУЛЬТАТОВ

5.1 Стандартные ассерты JUnit 5

```
1  import static org.junit.jupiter.api.Assertions.*;
2
3  @Test
4  void standardAssertions() {
5      // Равенство
6      assertEquals(7, calculator.sum(2, 5));
7      assertEquals("Expected", actualString, "Сообщение при ошибке");
8
9      // Неравенство
10     assertNotEquals(null, object);
11
12     // Логические значения
13     assertTrue(condition);
14     assertFalse(condition, "Сообщение при провале");
15
16     // Проверка на null
17     assertNull(object);
18     assertNotNull(object, "Объект не должен быть null");
19
20     // Проверка исключений
21     Exception exception = assertThrows(
22         IllegalArgumentException.class,
23         () -> calculator.divide(10, 0)
24     );
25     assertEquals("Division by zero", exception.getMessage());
26
27     // Группа ассертов (все выполняются даже при провале одного)
28     assertAll("Проверка пользователя",
29         () -> assertEquals("John", user.getName()),
30         () -> assertEquals(25, user.getAge()),
31         () -> assertNotNull(user.getEmail())
32     );
33 }
```

Листинг 5: Основные методы Assertions

5.2 AssertJ: более читаемые ассерты

Библиотека **AssertJ** предоставляет fluent API для более понятных проверок:

```
1  import static org.assertj.core.api.Assertions.*;
2
3  @Test
4  void assertJExamples() {
5      // Строки
6      assertThat("Hello World")
7          .isNotNull()
8          .startsWith("Hello")
9          .endsWith("World")
10         .contains("lo Wo");
11
12     // Коллекции
13     List<String> names = List.of("Alice", "Bob", "Charlie");
```

```
14     assertThat(names)
15         .isEmpty()
16         .hasSize(3)
17         .contains("Alice", "Bob")
18         .doesNotContain("David")
19         .allMatch(s -> s.length() > 2);
20
21     // Числа и сравнения
22     assertThat(10).isGreaterThan(5);
23     assertThat(3.14).isCloseTo(3.1415, within(0.01));
24
25     // BigDecimal (корректное сравнение!)
26     BigDecimal actual = new BigDecimal("1234.00100");
27     BigDecimal expected = new BigDecimal("1234.001");
28     assertThat(actual).isEqualByComparingTo(expected);
29
30     // Исключения
31     assertThatThrownBy(() -> calculator.divide(10, 0))
32         .isInstanceOf(ArithmeticException.class)
33         .hasMessageContaining("zero");
34
35     // Опциональные значения
36     Optional<String> optional = Optional.of("Value");
37     assertThat(optional)
38         .isPresent()
39         .hasValue("Value");
40 }
```

Листинг 6: Примеры AssertJ

Преимущества AssertJ:

- Читаемые сообщения об ошибках без дополнительных усилий;
- Цепочка вызовов (fluent API) улучшает читаемость;
- Богатый набор проверок для разных типов данных;
- Корректная работа с `BigDecimal`, коллекциями, исключениями.

6 ПАРАМЕТРИЗОВАННЫЕ ТЕСТЫ

Когда нужно проверить один и тот же метод с разными входными данными, используйте параметризованные тесты вместо дублирования кода.

6.1 Подключение

Убедитесь, что зависимость `junit-jupiter-params` добавлена в проект.

6.2 Аннотация `@ParameterizedTest`

```

1  import org.junit.jupiter.params.ParameterizedTest;
2  import org.junit.jupiter.params.provider.CsvSource;
3
4  @ParameterizedTest
5  @CsvSource({
6      "0, 0:00",
7      "10, 0:10",
8      "60, 1:00",
9      "95, 1:35",
10     "3665, 1:01:05"
11 })
12 @DisplayName("Конвертация секунд в формат мм:ss или ч:мм:ss")
13 void convertSecondsToTimeFormat(int seconds, String expected) {
14     String result = TimeConverter.format(seconds);
15     assertEquals(expected, result);
16 }
```

Листинг 7: Простой параметризованный тест с `@CsvSource`

6.3 Источники данных для параметров

Аннотация	Описание
<code>@CsvSource</code>	Передача простых значений через запятую. Поддерживает <code>String</code> , числа, <code>boolean</code> , <code>enum</code> , <code>Duration</code> и др.
<code>@ValueSource</code>	Передача одного параметра: <code>@ValueSource(ints = {1, 2, 3})</code>
<code>@MethodSource</code>	Ссылка на статический метод, возвращающий <code>Stream<Arguments></code> для сложных объектов.
<code>@EnumSource</code>	Передача значений <code>enum</code> .

Таблица 2: Источники данных для параметризованных тестов

6.4 Пример с `@MethodSource` для сложных объектов

```

1  static Stream<Arguments> provideSongsAndExpectedLength() {
2      return Stream.of(
3          Arguments.of(
4              Song.builder().duration(Duration.ofSeconds(0)).build(),
5              "0:00"
6          ),
7      );
8  }
```

```
7         Arguments.of(  
8             Song.builder()  
9                 .title("Bohemian Rhapsody")  
10                .duration(Duration.ofSeconds(354))  
11                .build(),  
12            "5:54"  
13        ),  
14        Arguments.of(  
15            Song.builder().duration(Duration.ofMinutes(10)).build(),  
16            "10:00"  
17        )  
18    );  
19 }  
20  
21 @ParameterizedTest  
22 @MethodSource("provideSongsAndExpectedLength")  
23 @DisplayName("Конвертация длительности песни в строковый формат")  
24 void songDurationToLength(Song song, String expectedLength) {  
25     DurationToLengthConverter converter = new  
26         DurationToLengthConverter(song);  
27     assertThat(converter.length()).isEqualTo(expectedLength);  
28 }
```

Листинг 8: Использование @MethodSource

7 ЛУЧШИЕ ПРАКТИКИ НАПИСАНИЯ ТЕСТОВ

7.1 Именование тестов

Имя теста должно отвечать на вопросы: *Что тестируем? При каких условиях? Какой ожидаем результат?*

```
1 // Плохо:
2 @Test
3 void test1() { ... }
4
5 @Test
6 void testUser() { ... }
7
8 // Хорошо:
9 @Test
10 @DisplayName("Возвращает true при валидном email")
11 void isValidEmail_WithValidFormat_ReturnsTrue() { ... }
12
13 @Test
14 @DisplayName("Бросает IllegalArgumentException при null-аргументе")
15 void createUser_WithNullEmail_ThrowsException() { ... }
```

7.2 Один тест — одна проверка

Каждый тест должен проверять **одну концепцию**. Это упрощает понимание причины провала.

```
1 // Не рекомендуется: несколько независимых проверок в одном тесте
2 @Test
3 void testCalculator() {
4     assertEquals(4, calc.sum(2, 2));
5     assertEquals(0, calc.sum(-1, 1));
6     assertEquals(-4, calc.sum(-2, -2));
7     // Если первый ассерт упадёт, остальные не выполнятся!
8 }
9
10 // Рекомендуется: отдельные тесты для разных сценариев
11 @Test
12 void sum_TwoPositiveNumbers_ReturnsSum() {
13     assertEquals(4, calc.sum(2, 2));
14 }
15
16 @Test
17 void sum_PositiveAndNegative_ReturnsCorrectResult() {
18     assertEquals(0, calc.sum(-1, 1));
19 }
20
21 @Test
22 void sum_TwoNegativeNumbers_ReturnsNegativeSum() {
23     assertEquals(-4, calc.sum(-2, -2));
24 }
```

7.3 Используйте @BeforeEach для общей подготовки

Избегайте дублирования кода инициализации:

```
1 class OrderServiceTest {
2     private OrderService orderService;
3     private PaymentGateway mockGateway;
4
5     @BeforeEach
6     void setUp() {
7         mockGateway = Mockito.mock(PaymentGateway.class);
8         orderService = new OrderService(mockGateway);
9     }
10
11     @Test
12     void createOrder_WithValidData_SavesOrder() { ... }
13
14     @Test
15     void createOrder_WithInvalidData_ThrowsException() { ... }
16 }
```

7.4 Изолируйте тесты от внешних зависимостей

Юнит-тесты не должны зависеть от:

- Базы данных;
- Сетевого доступа;
- Файловой системы;
- Времени выполнения.

Используйте моки (Mockito) или фейки для изоляции:

```
1 @Test
2 void processOrder_CallsPaymentGateway_WithCorrectAmount() {
3     // Arrange
4     PaymentGateway mockGateway = Mockito.mock(PaymentGateway.class);
5     OrderService service = new OrderService(mockGateway);
6     Order order = new Order(100.0);
7
8     // Act
9     service.processOrder(order);
10
11     // Assert
12     Mockito.verify(mockGateway).charge(100.0);
13 }
```

7.5 Пишите тесты до или параллельно с кодом (TDD)

Подход **Test-Driven Development**:

1. Напишите тест для новой функциональности (тест падает);
2. Напишите минимальный код, чтобы тест прошёл;
3. Рефакторите код, сохраняя прохождение тестов.

Преимущества:

- Код изначально проектируется тестируемым;
- Тесты покрывают все требования;
- Регрессии обнаруживаются сразу.

7.6 Поддерживайте тесты в актуальном состоянии

- Если тест падает — исправляйте **сразу**;
- Удаляйте дублирующиеся тесты;
- Рефакторите тесты так же, как продакшен-код;
- Не отключайте тесты без веской причины (и не забывайте про `@Disabled`).

8 ТИПИЧНЫЕ ОШИБКИ И КАК ИХ ИЗБЕЖАТЬ

Ошибка	Последствия	Решение
Тесты с зависимостями друг от друга	Невозможно запускать тесты по отдельности; хрупкие тесты	Каждый тест должен быть независимым; используйте <code>@BeforeEach</code> для изоляции
Проверка реализации вместо поведения	Тесты ломаются при рефакторинге, даже если функциональность не изменилась	Тестируйте публичный API и ожидаемое поведение, а не внутренние детали
Слишком сложные тесты	Трудно понять причину провала; сложно поддерживать	Разбивайте сложные сценарии на несколько простых тестов
Игнорирование граничных случаев	Баги в продакшене при нестандартных входных данных	Тестируйте: null, пустые значения, мин/макс значения, некорректные форматы
Отсутствие сообщений в ассертах	При провале теста непонятно, что пошло не так	Используйте перегруженные методы ассертов с сообщением или <code>AssertJ</code>

Таблица 3: Типичные ошибки при написании тестов

9 ЗАКЛЮЧЕНИЕ

Написание качественных unit-тестов — это инвестиция в стабильность и поддерживаемость кода. Они помогают:

- Быстро находить и исправлять ошибки;
- Рефакторить код без страха сломать существующую функциональность;
- Передавать знания новым членам команды через читаемые тесты;
- Сократить время на ручное тестирование и отладку.

«Хороший тест — это тест, который падает, когда в коде появляется баг, и не падает, когда код работает правильно.»

10 ДОПОЛНИТЕЛЬНЫЕ РЕСУРСЫ

- Официальная документация JUnit 5
- Документация AssertJ
- Mockito для создания моков
- Книга: *"Test Driven Development: By Example"* — Kent Beck